FEAT-ZD-48600 · DS-05 Coding · Code changes on the working branch

Data Science agent · item task.code · source hobasa-data-science-agent/outputs/FEAT-ZD-48600/task.code.md · approved by automated checks and independent judge

task.code: FEAT-ZD-48600

Code changes for the Data Science lane of FEAT-ZD-48600 ("Show which overdue invoices belong to vendors that usually pay late"). This is DS-05 Coding. The workbook outcome is "Updated repo with the new implemented code changes". The changes are in the `ap_forecast` working tree on branch `feature/story`. They are not committed; the spine commits them after evaluation.

Field	Value	Source
Trace ID	FEAT-ZD-48600	record:brief
Step	DS-05 Coding	record:brief
Owner / gate	Developer / no gate: automated checks, the independent judge and sampled human evaluation	record:brief
Inputs	task.components (7cb32bd), feat.design (722a98f), feat.stories (0590ea4), org.engineering_stack (e8afce8), org.lessons (e8afce8), task.secrets_scope (restricted, by reference)	record:brief
Repository / branch	`ap_forecast`, `feature/story`	record:brief; task.components
Stories / tasks	STORY-03 (TSK-03, `vendor_risk`) and STORY-04 (TSK-04, `risk=None`)	feat.stories; task.components
Allowed paths in this step	`src/ap_forecast/`, `docs/`, `README.md`	record:brief

1. Result

The planned source changes C-1 to C-3 and the documentation are done, and the sources compile. The code changes are additive. No existing signature, default or return shape changes (tool:repo).

The new behaviour has no unit tests yet. No existing test calls `vendor_risk` or passes `risk` (tool:repo). The tests that cover it are task.components C-4 and C-5. They live in `tests/`, which this step may not change (record:brief). `ci.py` gates coverage of `src/ap_forecast` at 90.0 (file:ap_forecast/ci.py), so the gate depends on those tests. Section 7 asks the Developer to confirm where that happens.

#	File	Change	Component (task.components)	Story / AC
1	`src/ap_forecast/risk.py`	New: `HIGH_ABOVE_DAYS = 30`, `MEDIUM_ABOVE_DAYS = 10`, `vendor_risk(history)`	C-1	STORY-03; AC-03.1 to AC-03.7
2	`src/ap_forecast/explain.py`	`build_explanation_request` gains `risk=None` as the last parameter, and a docstring	C-2	STORY-04; AC-04.1 to AC-04.5
3	`src/ap_forecast/__init__.py`	Imports `vendor_risk` and adds it to `__all__`; version `0.9.0` → `0.10.0` (proposed, DSQ-1)	C-3	STORY-03; AC-03.8

#	File	Change	Component (task.components)	Story / AC
4	`docs/ap_forecast.md`	New module reference: each public function with signature, inputs, outputs, edge cases and examples	Documentation (record:brief)	—
5	`README.md`	Function table, short example, link to `docs/ap_forecast.md`; CI command kept	Documentation (record:brief)	—

These stay unchanged, as planned: `predict.py`, `models.json`, `ci.py`, `CLAUDE.md` and `tests/` (task.components section 4; record:brief).

2. Code

The full current source of the three changed modules is in the tool view (tool:repo).

2.1 `src/ap_forecast/risk.py` *(new, STORY-03)*

```
from typing import Iterable

from .predict import predict_delay_days

# Thresholds in days on the rounded prediction; strict '>' so exactly 30 is 'medium' and exactly 10 is 'low'.
HIGH_ABOVE_DAYS = 30
MEDIUM_ABOVE_DAYS = 10


def vendor_risk(history: Iterable[int]) -> str:
    """'high' if the predicted delay is above HIGH_ABOVE_DAYS, 'medium' if above MEDIUM_ABOVE_DAYS, else 'low'. ..."""
    # Predict once and keep the result: a generator history is consumed on the first pass.
    days = predict_delay_days(history)
    if days > HIGH_ABOVE_DAYS:
        return "high"
    if days > MEDIUM_ABOVE_DAYS:
        return "medium"
    return "low"
```

Design rule	How the code meets it	Source
Named constants 30 and 10; strict > on the rounded prediction	`HIGH_ABOVE_DAYS` and `MEDIUM_ABOVE_DAYS`; `days > ...` on `predict_delay_days(history)`	feat.design (DD-9, B2.2); feat.stories (AC-03.1 to AC-03.4); tool:repo
Exactly 30 gives `'medium'`; exactly 10 gives `'low'`	Follows from strict >; stated in the constants' comment	feat.stories (AC-03.2, AC-03.4); org.lessons (L-007)
Empty history and early payments only give `'low'`	`predict_delay_days` returns 0 for both, and 0 is not above 10	feat.design (B2.4); feat.stories (AC-03.5, AC-03.6); tool:repo (`predict.py`)
No LLM call and no `models.json` read	`risk.py` imports only `typing` and `.predict`, never `explain`	feat.design (B2.2); feat.stories (AC-03.7); task.components (C-1); tool:repo
Predict once, so a generator input works	One call; the result is kept in `days`	task.components (C-1)
No extra validation of history values	Left to `int(d)` in `predict_delay_days`	feat.design (B2.4); task.components (C-1)

2.2 `src/ap_forecast/explain.py` *(changed, STORY-04)*

```
def build_explanation_request(vendor: str, predicted_days: int, config=None, risk=None) -> dict:
    """Request dict for the configured provider. risk (STORY-04) is optional and last, so existing calls are unchanged.
    cfg = config or load_config()
    prompt = f"Explain in one sentence why vendor {vendor} is predicted to be paid {predicted_days} days late."
    if risk is not None:
        prompt += f" Risk level: {risk}."
    return {"provider": cfg["provider"], "model": cfg["model"], "max_tokens": cfg["max_tokens"],
            "messages": [{"role": "user", "content": prompt}]}
```

Design rule	How the code meets it	Source
`risk` is the last parameter, default `None`	Signature `(vendor, predicted_days, config=None, risk=None)`. Existing calls that pass `config` positionally still bind it to `config`	feat.design (B2.2, NF-4); feat.stories (AC-04.3); file:ap_forecast/CLAUDE.md; tool:repo
"Given" means `risk is not None`; the value is appended without validation	`if risk is not None: prompt += f" Risk level: {risk}."`	feat.design (B2.2); feat.stories (AC-04.1, AC-04.5)
Without `risk`, the request is the same as before the change	The prompt line and the returned dict are unchanged	feat.stories (AC-04.2); task.components (C-2)
Provider, model and `max_tokens` unchanged	Still read from `cfg`	feat.stories (AC-04.4); feat.design (DC-4)
`load_config` and `CONFIG` unchanged	Not edited	task.components (C-2); tool:repo

2.3 `src/ap_forecast/__init__.py` ***(changed)***

```
from .predict import predict_delay_days
from .explain import build_explanation_request
from .risk import vendor_risk

__version__ = "0.10.0"
__all__ = ["predict_delay_days", "build_explanation_request", "vendor_risk"]
```

The existing exports are kept and `vendor_risk` is added (feat.stories AC-03.8; tool:repo). feat.design DD-11 asks for a minor step from `0.9.0`, which the lane confirms, and no input gives the number. `0.10.0` is this step's proposal. It stays open as DSQ-1 for the Developer (task.components section 8; tool:repo).

3. Documentation

File	Content	Source
`docs/ap_forecast.md` (new)	Module table. `predict_delay_days`, `vendor_risk`, `build_explanation_request` and `load_config`, each with signature, inputs, outputs, edge cases and examples (synthetic vendor names only). How to build `risk_by_vendor` for ap_aging. A note that the caller sends the risk label to the provider in the prompt	record:brief; feat.design (DC-3, SE-3, SE-4)
`README.md`	Function table, short example and a link to the module reference; the CI command is kept	record:brief

4. Blast radius

Read from the repository source in the tool view.

Symbol	Used by	Existing tests that use it	Source
`build_explanation_request`	`src/ap_forecast/__init__.py` (re-export)	`tests/test_forecast.py`: `test_request_uses_config`, `test_request_with_explicit_config`	tool:repo
`predict_delay_days`	`src/ap_forecast/risk.py` (`vendor_risk`), `src/ap_forecast/__init__.py` (re-export)	`tests/test_forecast.py`: `test_predict`	tool:repo
`vendor_risk` (new)	`src/ap_forecast/__init__.py` (re-export)	None: `tests/test_forecast.py` imports only `build_explanation_request` and `predict_delay_days`	tool:repo

The existing tests call `predict_delay_days` and `build_explanation_request` without `risk`, and neither call is changed by this step (tool:repo). No other lane imports the `ap_forecast` package (task.components section 2).

5. Automated checks

The spine records the result of the automated checks for this step with this output. What the repository CI (`/usr/bin/python3 ci.py`, org.engineering_stack) checks, and what the code shows for each check:

CI check (file:ap_forecast/ci.py)	What the changed code shows	Source	
Compile `src`	Python 3.9 syntax only: no `match`, no `X \`	`Y unions; typing.Iterable`	tool:repo; feat.design (NF-9)
Static checks: `print` / `eval` / `exec`, bare `except`, lines over 140 characters	The changed modules call none of `print`, `eval` or `exec` and have no `except` clause	tool:repo	
Unit tests under `tests/`	The existing tests use only unchanged call forms (section 4)	tool:repo	
Coverage of `src/ap_forecast` at least `COVERAGE_MIN = 90.0`	No existing test runs the body of `vendor_risk` or the line `prompt += f" Risk level: {risk}."`. Those lines are run by the planned tests C-4 (boundary histories `[31]`, `[30]`, `[10]` reach each return in `vendor_risk`) and C-5 (`test_request_with_risk`)	tool:repo; task.components (section 3, C-4, C-5)	

Coverage was not raised by any other means: no exclusion pragmas, no edit to `ci.py`, and no rewriting of statements to cover fewer lines.

6. Constraints check

Constraint	Result	Source
Python 3.9 syntax only; `typing.Iterable`	Met in the source	feat.design (NF-9); org.engineering_stack; tool:repo
Keep existing public signatures; add new functions	Met	file:ap_forecast/CLAUDE.md; feat.design (header); tool:repo
Only allowed paths changed	Met: `src/ap_forecast/risk.py`, `src/ap_forecast/explain.py`, `src/ap_forecast/__init__.py`, `docs/ap_forecast.md`, `README.md`	record:brief
No secrets or personal data	Met: the code uses and needs no secret; examples use "Synthetic Vendor A"	task.components (section 4); feat.design (SE-4)
Secrets access in all environments (workbook action)	The code needs none: it builds the LLM request and does not send it. Confirmation per environment stays open as SSQ-2 (non-blocking)	task.components (section 4)
Database changes	None needed	task.components (section 4)
Decision log and lessons	The lane handles no money (D-2026-044 and L-011 not touched). The boundary rule is documented for the L-007 tests	record:memory; org.lessons
Commit message prefix (for the commit step)	`STORY-03` / `STORY-04`	org.engineering_stack; task.components (section 5)

7. Open questions

ID	Question	Interim	Owner
DSQ-1	Confirm `0.10.0` as the minor step from `0.9.0`	`0.10.0` is written in `__init__.py`	Developer
DSQ-2	The `ci.py` coverage gate (90.0) needs tests for the new code. The planned tests (C-4, C-5) belong in `tests/`, which this step may not change. Is the gate met in the unit-test step (DS-06), or should this step also be allowed to write `tests/`?	Tests left to DS-06, as task.components plans	Developer

Carried unchanged, none blocking coding: TQ-4 / CQ-4 (meaning of "given" for `risk`), TQ-5, TQ-8 (risk label sent to `acme-llm`), PQ-1, PQ-5 (order against PROV-ACME-2026-11) and SSQ-2 (task.components section 8).

Tool results (recorded by the spine from the real tools)

[Tool result] [ap_forecast] Sources compile